7. At this point, you can also examine app.js, which serves as the glue between the web UI and the smart contract. This critical component in the end-to-end design of a Dapp handles the calls from the UI and directs them to the smart contract.

You've completed a simple positive test. You can explore the ASK Dapp further on your own to see how it handles reversions and negative tests.

Finally, you may be wondering about the use of the `ASKRequest()` and `ASKResponse()` operations, which don't do any computations. They merely store the parameters to the smart contract variables, creating on-chain records or the state of what transpired off-chain. This action ensures that the valid transactions and state corresponding to off-chain requests and responses are recorded on the blockchain. You can view these Txs on Etherscan or Ganache. The goal of this chapter is to make sure the data about the valid Txs is recorded and payments are settled automatically.

6.5 Retrospective

This chapter brought out an important distinction between traditional programming and blockchain programming: you need to design the on-chain data (the data that gets recorded on the blockchain) carefully to avoid overloading the blockchain. You can use the types of on-chain data introduced and discussed in this chapter (receipts and event logs, transaction data, and state data) as guidelines when designing smart contracts. The blind auction Dapp focused only on event-based on-chain data. ASK Dapp illustrated the use of transaction and state-related on-chain data. The state on-chain data records how the variables in